



Politechnika  
Wrocławska

Wydział Informatyki i Telekomunikacji  
Kierunek: Informatyka Techniczna

# Projektowanie i programowanie gier

## Raport z części podstawowej laboratorium

Imię i nazwisko: Wiktor Zięba  
Numer albumu: 281056

Wrocław, czerwiec 2025

# Spis treści

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Wprowadzenie</b>                                     | <b>2</b>  |
| <b>2</b> | <b>Gra 2D</b>                                           | <b>2</b>  |
| 2.1      | Utworzenie projektu i konfiguracja okna . . . . .       | 2         |
| 2.2      | Przygotowanie sceny gracza . . . . .                    | 2         |
| 2.3      | Sterowanie i kod gracza . . . . .                       | 3         |
| 2.4      | Tworzenie przeciwnika . . . . .                         | 5         |
| 2.5      | Główna scena gry . . . . .                              | 7         |
| 2.6      | HUD . . . . .                                           | 9         |
| 2.7      | Efekt końcowy . . . . .                                 | 10        |
| <b>3</b> | <b>Gra 3D</b>                                           | <b>11</b> |
| 3.1      | Przygotowanie podstawowej sceny 3D . . . . .            | 11        |
| 3.2      | Dodanie kuli gracza . . . . .                           | 13        |
| 3.3      | Śledzenie gracza przez kamerę . . . . .                 | 14        |
| 3.4      | Importowanie tekstur i zmiana wyglądu planszy . . . . . | 15        |
| 3.5      | Budowa poziomu z użyciem <code>GridMap</code> . . . . . | 17        |
| 3.6      | Przeciwnicy i animacje ruchu . . . . .                  | 17        |
| 3.7      | Dodanie monet, HUD-u oraz ekranów końcowych . . . . .   | 18        |
| 3.8      | Efekt końcowy projektu 3D . . . . .                     | 20        |
| <b>4</b> | <b>Źródła</b>                                           | <b>20</b> |

# 1 Wprowadzenie

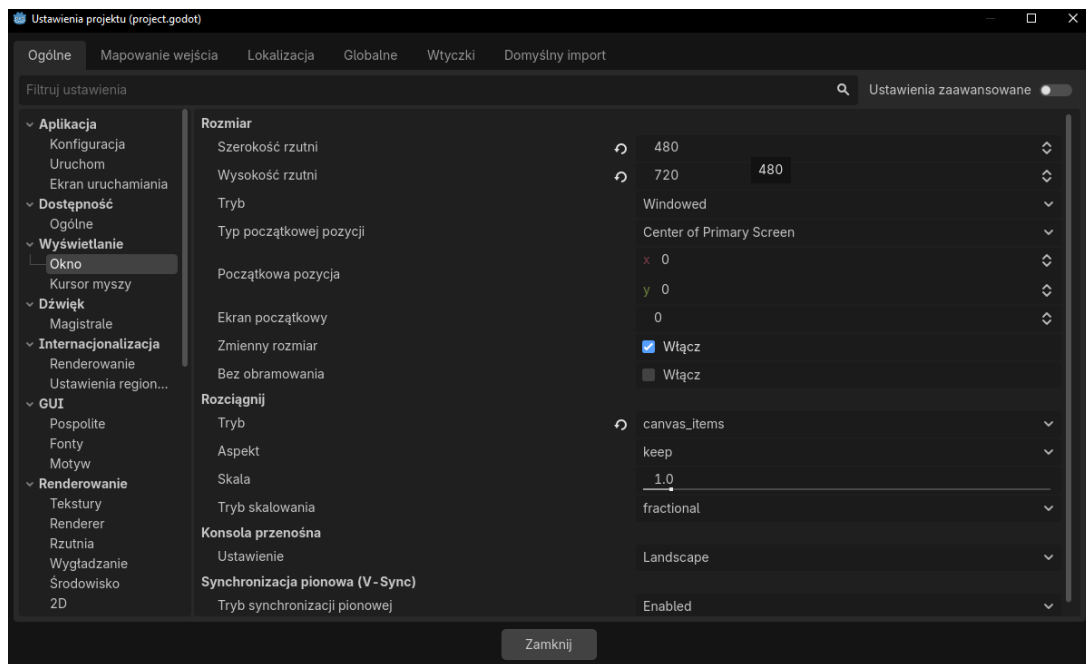
Celem laboratorium było zapoznanie się z podstawami pracy w silniku **Godot** poprzez wykonanie dwóch prostych projektów: gry 2D zgodnie z oficjalnym poradnikiem oraz gry 3D według tutoriala z kanału BornCG na youtube. Praca była realizowana krok po kroku według kolejnych części dokumentacji i filmów. W ramach ćwiczenia powstały dwie w pełni grywalne i zgodne z poradnikami gry. Projekty zostały wykonane w środowisku Godot 4.6.1.

## 2 Gra 2D

Pierwsza część laboratorium polegała na wykonaniu prostej gry 2D zgodnie z oficjalnym poradnikiem Godot. Celem gry jest poruszanie się gracza po planszy i unikanie przeciwników pojawiających się na ekranie.

### 2.1 Utworzenie projektu i konfiguracja okna

Pracę rozpoczęto od utworzenia nowego projektu oraz zaimportowania gotowych zasobów graficznych i dźwiękowych dostarczonych w tutorialu. Następnie ustawiono rozmiar obszaru gry na **480 × 720 px** oraz skonfigurowano sposób skalowania obrazu tak, żeby gra zachowywała poprawne proporcje po uruchomieniu w oknie. Na tym etapie przygotowano również strukturę folderów projektu, tak aby osobno przechowywać grafiki, czcionki, sceny i skrypty.



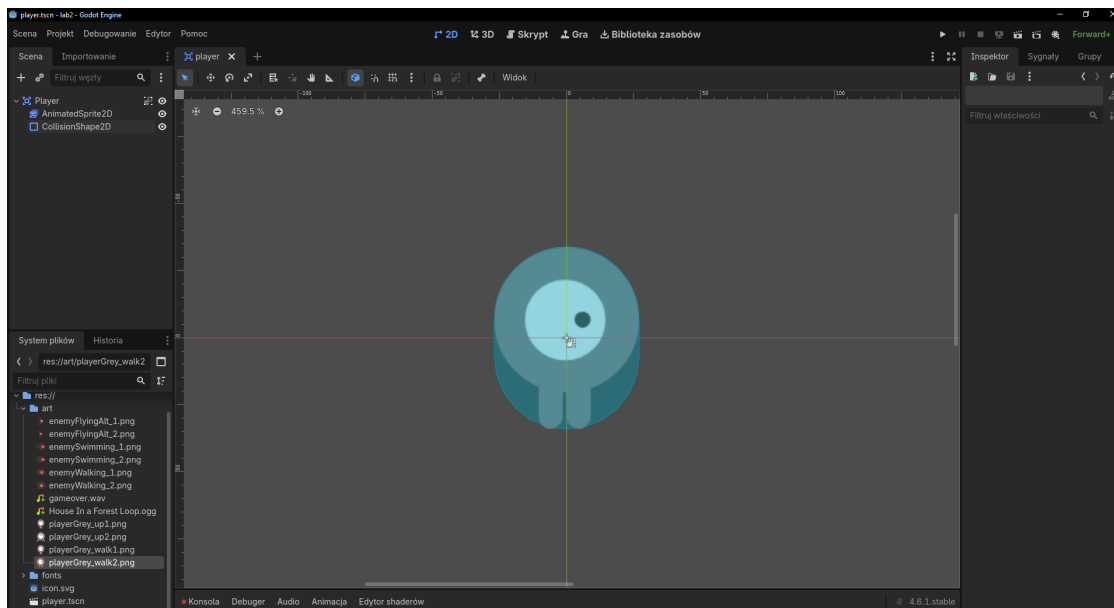
Rysunek 1: Ustawienia projektu – konfiguracja rozmiaru okna i skalowania.

### 2.2 Przygotowanie sceny gracza

Scenę gracza zbudowano na bazie węzła **Area2D** o nazwie **Player**. Do niego dodano dwa podstawowe elementy potomne:

- **AnimatedSprite2D** – odpowiadający za wygląd i animacje postaci
- **CollisionShape2D** – odpowiadający za wykrywanie kolizji

Dla postaci przygotowano animacje **walk** oraz **up**. Dzięki temu postać gracza ma różne animacje poruszania się w poziomie i pionie. Kształt kolizji dopasowano do kształtu modelu gracza. Użyto w tym celu odpowiednio przeskalowany kształt **CapsuleShape**.



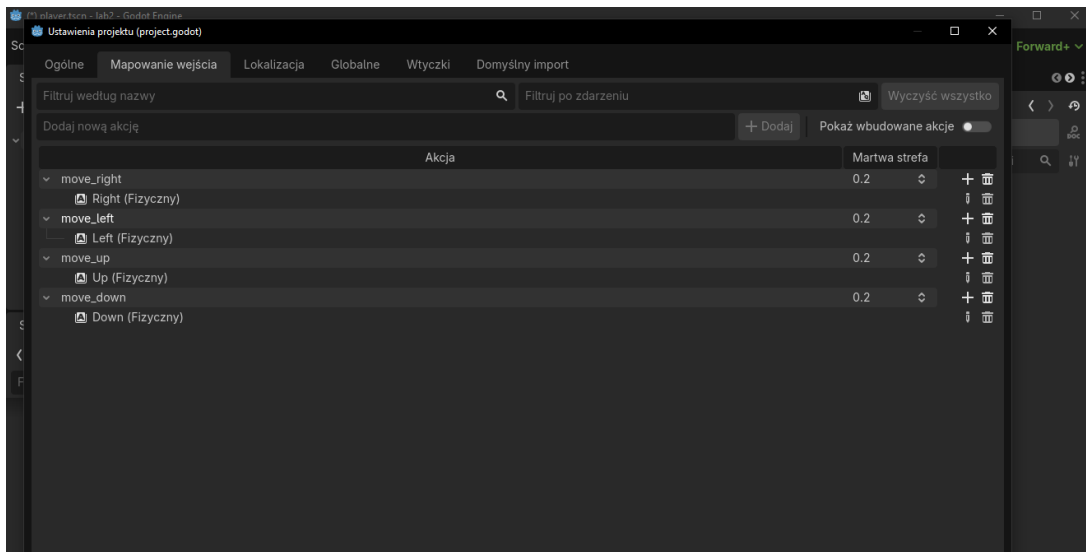
Rysunek 2: Dopasowanie kształtu kolizji do modelu gracza

## 2.3 Sterowanie i kod gracza

Aby postać mogła się poruszać, skonfigurowano własne akcje wejściowe: **move\_left**, **move\_right**, **move\_up** oraz **move\_down**. Zostały one przypisane do klawiszy strzałek. Dzięki takiemu rozwiązaniu jest możliwe odwoływanie się skryptu do nazw akcji zamiast konkretnych klawiszy. Kod gracza realizuje następujące zadania:

- odczytuje wciśnięte akcje wejściowe
- wyznacza wektor prędkości ruchu
- ogranicza położenie postaci do rozmiaru okna
- uruchamia animacje
- emituje sygnał **hit** po wykryciu kolizji z przeciwnikiem





Rysunek 3: Konfiguracja akcji wejściowych w zakładce Input Map.

```

1 extends Area2D
2 signal hit
3
4 @export var speed = 400
5 var screen_size: Vector2
6
7 func _ready() -> void:
8     screen_size = get_viewport_rect().size
9     hide()
10
11
12
13 func _process(delta: float) -> void:
14     var velocity = Vector2.ZERO
15     if Input.is_action_pressed("move_right"):
16         velocity.x+=1
17     if Input.is_action_pressed("move_left"):
18         velocity.x-=1
19     if Input.is_action_pressed("move_up"):
20         velocity.y-=1
21     if Input.is_action_pressed("move_down"):
22         velocity.y+=1
23
24     if velocity.length() > 0:
25         velocity = velocity.normalized() * speed
26         $AnimatedSprite2D.play()
27     else:
28         $AnimatedSprite2D.stop()
29

```

```

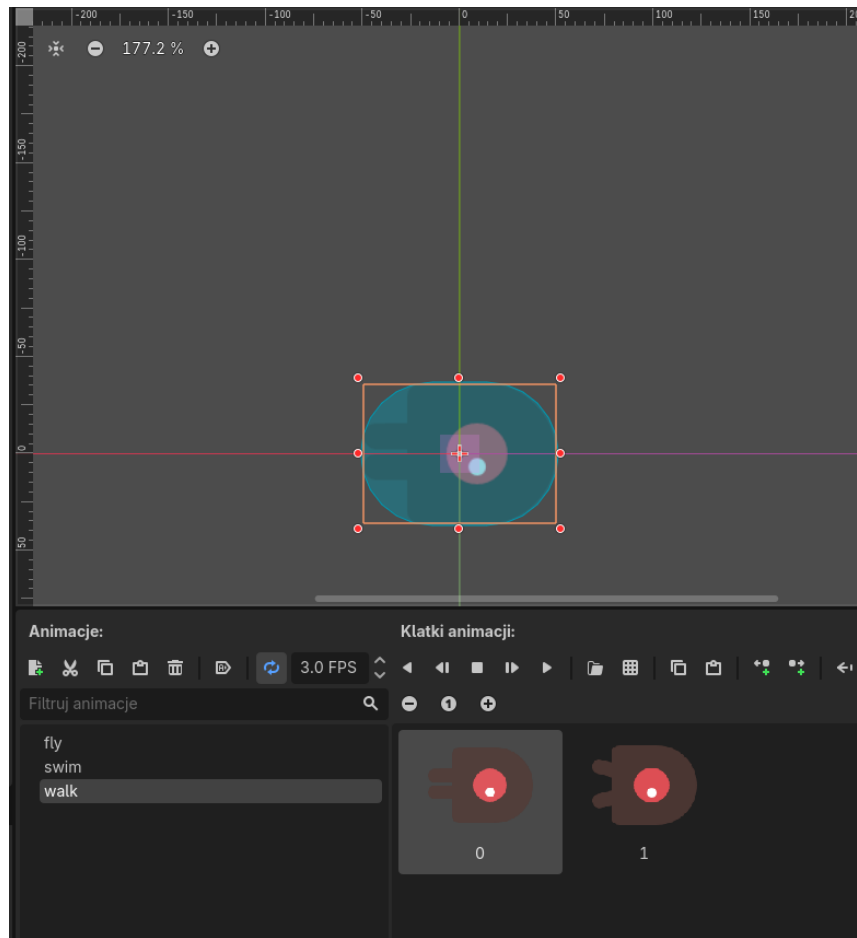
30     position += velocity * delta
31     position = position.clamp(Vector2.ZERO, screen_size)
32
33
34     if velocity.x != 0:
35         $AnimatedSprite2D.animation = "walk"
36         $AnimatedSprite2D.flip_v = false
37         $AnimatedSprite2D.flip_h = velocity.x < 0
38     elif velocity.y != 0:
39         $AnimatedSprite2D.animation = "up"
40         $AnimatedSprite2D.flip_v = velocity.y > 0
41
42
43 func _on_body_entered(body: Node2D) -> void:
44     hide()
45     hit.emit()
46     $CollisionShape2D.set_deferred("disabled", true)
47
48 func start(pos):
49     position = pos
50     show()
51     $CollisionShape2D.disabled = false

```

Tekst 1: Pełny kod skryptu player.gd.

## 2.4 Tworzenie przeciwnika

Przeciwnik został zaprojektowany osobnej scenie Mob, której głównym węzłem jest RigidBody2D. Analogicznie jak w przypadku gracza, dodano animowany sprite i kształt kolizji. Dodatkowo do sceny dołączono VisibleOnScreenEnabler2D, dzięki któremu przeciwnik automatycznie usuwa się po opuszczeniu ekranu. Skrypt przeciwnika losuje jedną z dostępnych animacji i zaczyna ją odtwarzać po jego pojawieniu się. Gdy przeciwnik wyleci poza obszar gry, wywoływana jest funkcja `queue_free()`, która usuwa go z pamięci.



Rysunek 4: Animacje przeciwnika fly, swim oraz walk.

```

1 extends RigidBody2D
2
3 func _ready():
4     var mob_types = Array($AnimatedSprite2D.sprite_frames.
5         get_animation_names())
6     $AnimatedSprite2D.animation = mob_types.pick_random()
7     $AnimatedSprite2D.play()
8
9 func _on_visible_on_screen_notifier_2d_screen_exited():
10     queue_free()
11
12 func _process(delta: float) -> void:
13     pass

```

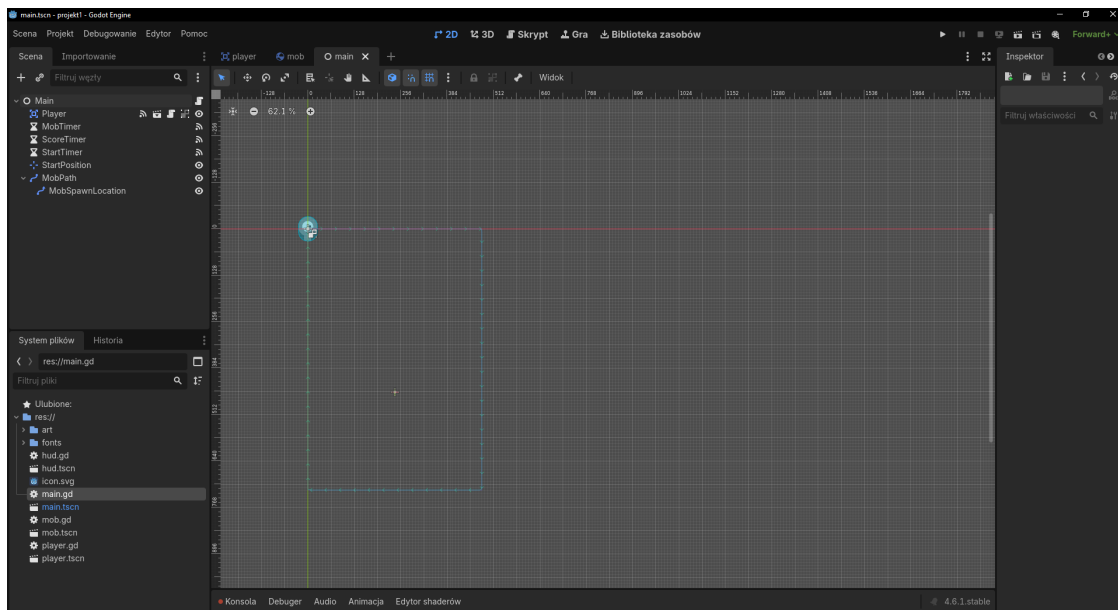
Tekst 2: Pełny kod skryptu mob.gd.

## 2.5 Główna scena gry

Głównym etapem projektowania gry było zaprojektowanie sceny **Main**, która zarządza przebiegiem całej rozgrywki. Zawiera ona instancję gracza, trzy timery, znacznik pozycji startowej oraz ścieżkę **MobPath**, po której wyznaczane są miejsca pojawiania się przeciwników. Logika sceny głównej obejmuje:

- rozpoczęcie nowej gry
- zatrzymanie gry po kolizji
- okresowe tworzenie nowych przeciwników
- aktualizację wyniku
- obsługę dźwięku tła i dźwięku porażki

Najistotniejsza funkcja w scenie to `_on_mob_timer_timeout()`. Tworzy ona nowego przeciwnika, losuje miejsce na ścieżce **MobPath**, wyznacza kierunek ruchu oraz nadaje mu losową prędkość. Dodatkowo na początku nowej gry usuwane są wszystkie stare obiekty należące do grupy **mobs**, żeby plansza była czyszczona przed kolejną rozgrywką.



Rysunek 5: Scena Main z widoczną ścieżką generowania przeciwników.

```
1 extends Node
2
3 @export var mob_scene: PackedScene
4 var score
5
6 func _ready() -> void:
7     pass
8
9 func _process(delta: float) -> void:
10    pass
```

```

11
12
13 func game_over():
14     $ScoreTimer.stop()
15     $MobTimer.stop()
16     $HUD.show_game_over()
17     $Music.stop()
18     $DeathSound.play()
19
20 func new_game():
21     get_tree().call_group("mobs", "queue_free")
22     score = 0
23     $Player.start($StartPosition.position)
24     $StartTimer.start()# Replace with function body.
25     $HUD.update_score(score)
26     $HUD.show_message("Get Ready")
27     $Music.play()
28
29
30
31
32 func _on_mob_timer_timeout() -> void:
33     var mob = mob_scene.instantiate()
34
35     var mob_spawn_location = $MobPath/MobSpawnLocation
36     mob_spawn_location.progress_ratio = randf()
37
38     mob.position = mob_spawn_location.position
39
40     var direction = mob_spawn_location.rotation + PI / 2
41
42     direction += randf_range(-PI / 4, PI / 4)
43     mob.rotation = direction
44
45     var velocity = Vector2(randf_range(150.0, 250.0), 0.0)
46     mob.linear_velocity = velocity.rotated(direction)
47
48     add_child(mob)
49
50
51 func _on_score_timer_timeout() -> void:
52     score += 1
53     $HUD.update_score(score)
54
55 func _on_start_timer_timeout() -> void:
56     $MobTimer.start()
57     $ScoreTimer.start()

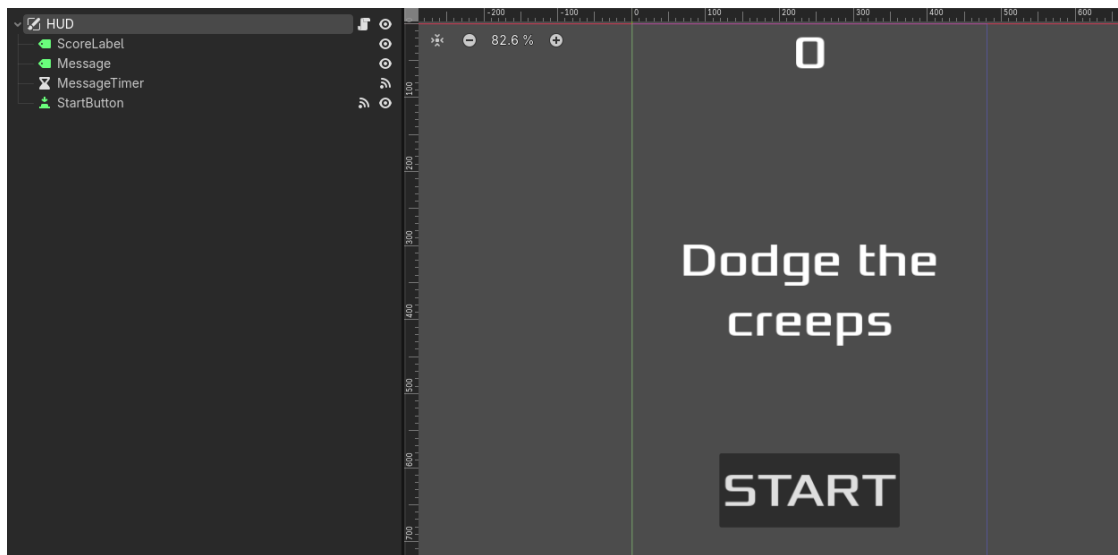
```

---

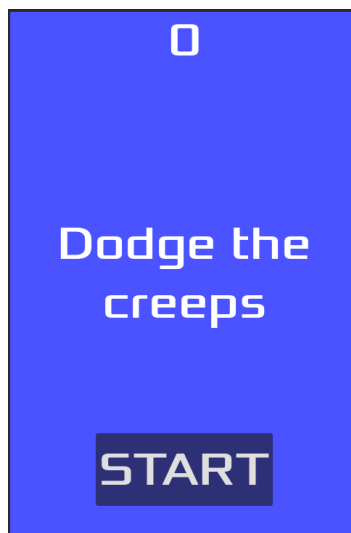
Tekst 3: Pełny kod skryptu `main.gd`.

## 2.6 HUD

Kolejnym etapem było przygotowanie interfejsu użytkownika. W tym celu utworzono scenę HUD jako węzeł `CanvasLayer`. Dzięki temu wszystkie elementy interfejsu są rysowane ponad planszą. HUD zawiera etykietę z wynikiem, komunikatów, przycisk start i timer odpowiedzialny za czas wyświetlania komunikatu. Skrypt HUD odpowiada za zmianę napisu z wynikiem, pokazywanie komunikatów oraz wyemitowanie sygnału `start_game` po naciśnięciu przycisku.



Rysunek 6: Scena HUD



Rysunek 7: Układ elementów na HUD

```

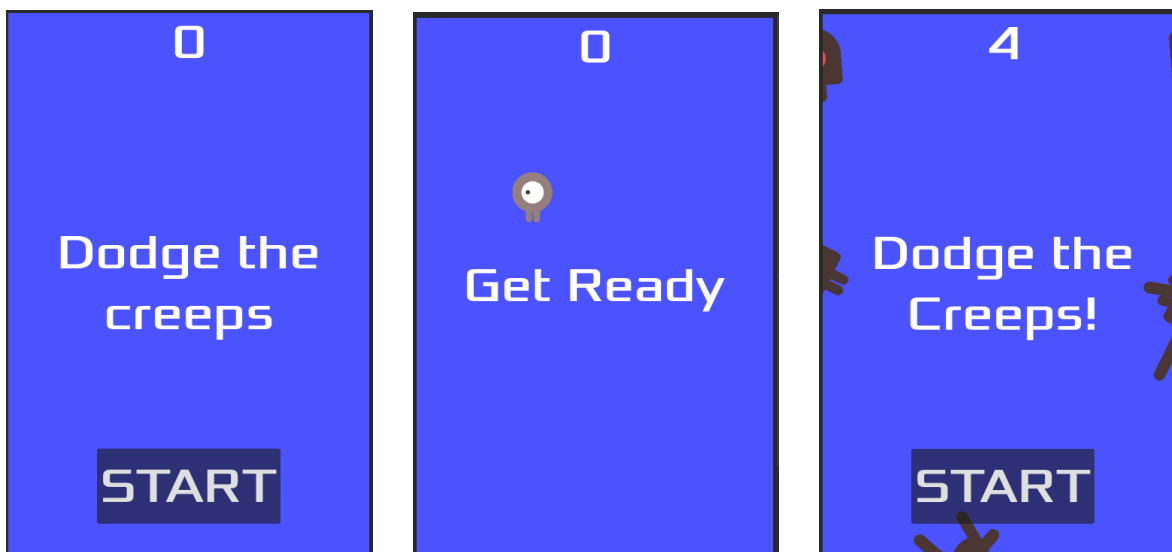
1 extends CanvasLayer
2
3 signal start_game
4
5 func show_message(text):
6     $Message.text = text
7     $Message.show()
8     $MessageTimer.start()
9
10 func _ready() -> void:
11     pass # Replace with function body.
12
13 func show_game_over():
14     show_message("Game Over")
15
16     await $MessageTimer.timeout
17
18     $Message.text = "Dodge the Creeps!"
19     $Message.show()
20
21     await get_tree().create_timer(1.0).timeout
22     $StartButton.show()
23
24 func update_score(score):
25     $ScoreLabel.text = str(score)
26
27 func _on_start_button_pressed():
28     $StartButton.hide()
29     start_game.emit()
30
31 func _on_message_timer_timeout():
32     $Message.hide()
33 # Called every frame. 'delta' is the elapsed time since the
34 # previous frame.
35 func _process(delta: float) -> void:
36     pass

```

Tekst 4: Pełny kod skryptu hud.gd.

## 2.7 Efekt końcowy

W ostatnim kroku na scenie głównej umieszczono `ColorRect`, tworząc jednolite tło, a także dodano węzły audio odpowiedzialne za muzykę w tle oraz dźwięk odtwarzany po przegranej. Po uruchomieniu rozgrywki przeciwnicy pojawiają się losowo na obrzeżach planszy, poruszają się w kierunku środka, a licznik punktów rośnie wraz z czasem przeżycia. Po kolizji gra zatrzymuje timery, wyświetla komunikat o końcu rozgrywki i umożliwia zagrania ponownie.



Rysunek 8: Efekt końcowy gry

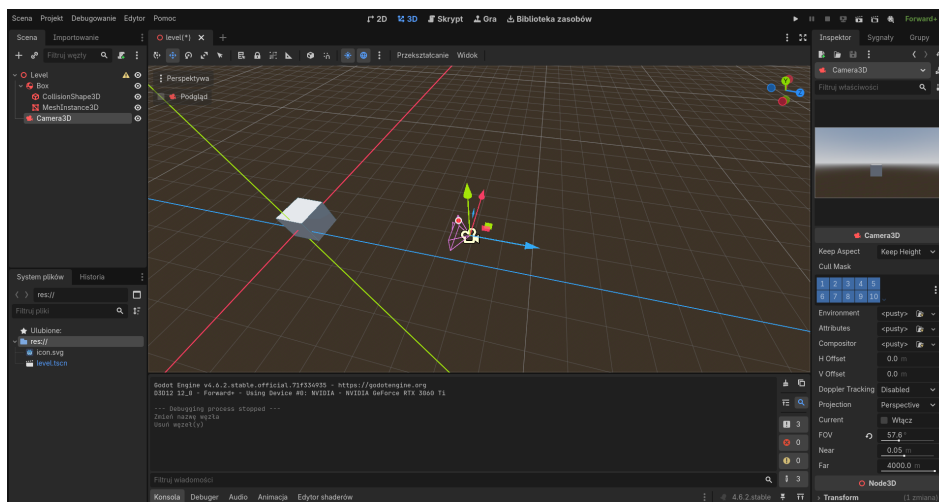
## 3 Gra 3D

Druga część laboratorium polegała na wykonaniu prostego projektu 3D na podstawie kolejnych materiałów wideo. W tej wersji gry gracz steruje kulą poruszającą się po planszy i jego celem jest zebranie wszystkich 10 monet nie zostając przy tym dotkniętym przez wrogą kulę.

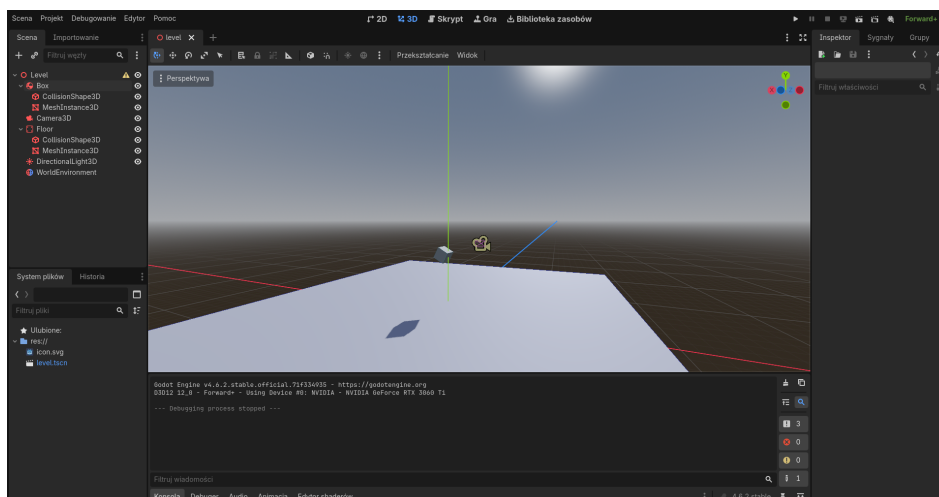
### 3.1 Przygotowanie podstawowej sceny 3D

Pierwszy odcinek poradnika skupiał się głównie na zaprezentowaniu interfejsu Godota, sposobu poruszania się w edytorze i sterowania. W ramach tego etapu stworzono również pierwsze testowe obiekty. Pracę rozpoczęto od utworzenia nowej sceny `Level`. Na początku dodano prosty obiekt testowy `Box` złożony z `CollisionShape3D` oraz `MeshInstance3D`, a także kamerę `Camera3D`. Następnie rozszerzono scenę o podłogę `Floor` będącą obiektem typu `StaticBody`. Dodatkowo do sceny dodano `DirectionalLight3D` oraz `WorldEnvironment`, aby oświetlenie wyglądało podobnie jak w poradniku. Początkowe różnice mogły być spowodowane realizacją projektu na nowszej wersji Godota.

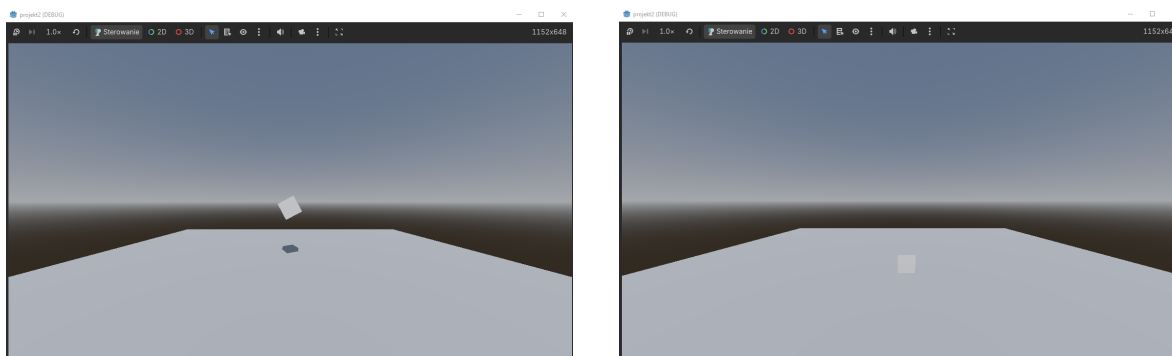




Rysunek 9: Wstępna faza tworzenia sceny



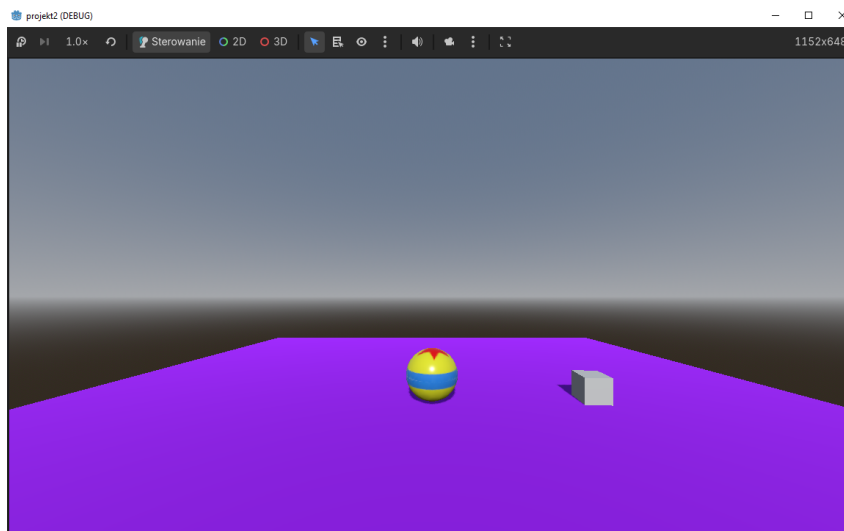
Rysunek 10: Scena po dodaniu podłogi i oświetlenia



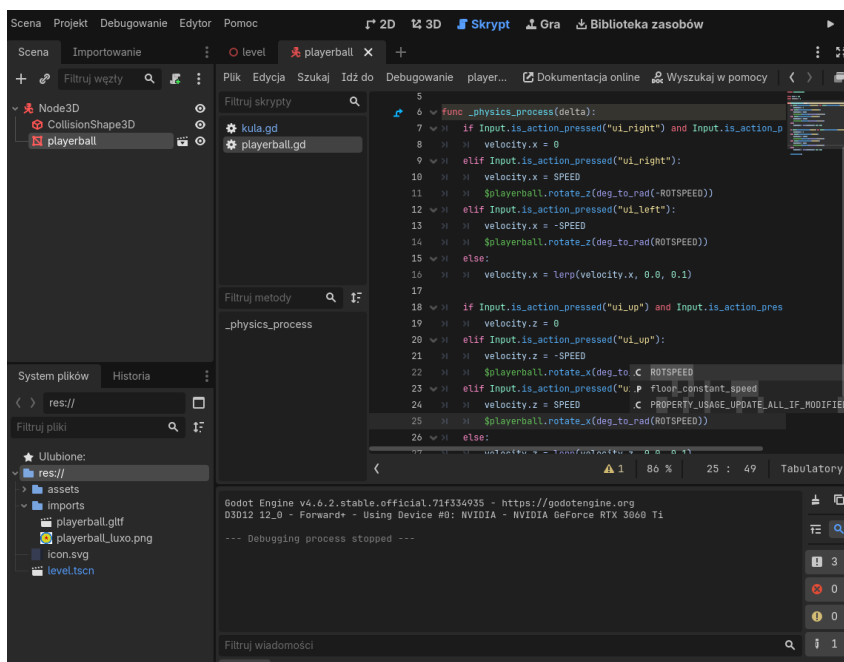
Rysunek 11: Uruchomienie stworzonej sceny i obserwacja fizyki obiektów

## 3.2 Dodanie kuli gracza

Kolejnym etapem było utworzenie obiektu gracza w postaci kuli. Do sceny kuli dodano zgodnie z poradnikiem kształt kolizji `BoxShape` oraz zaimportowany model. Następnie przygotowano skrypt odpowiedzialny za poruszanie kulą po planszy. Ruch został powiązany z wejściem z klawiatury, dzięki czemu obiekt może przemieszczać się po osi poziomej i pionowej planszy.



Rysunek 12: Obiektu gracza widoczny podczas działania gry



Rysunek 13: Implementacji skryptu odpowiedzialnego za ruch kuli

```

1 extends CharacterBody3D
2
3 const SPEED = 5.0
4
5 func _physics_process(delta):
6     if Input.is_action_pressed("ui_right") and Input.
7         is_action_pressed("ui_left"):
8         velocity.x = 0
9     elif Input.is_action_pressed("ui_right"):
10        velocity.x = SPEED
11    elif Input.is_action_pressed("ui_left"):
12        velocity.x = -SPEED
13    else:
14        velocity.x = lerp(velocity.x, 0.0, 0.1)
15
16    if Input.is_action_pressed("ui_up") and Input.
17        is_action_pressed("ui_down"):
18        velocity.z = 0
19    elif Input.is_action_pressed("ui_up"):
20        velocity.z = -SPEED
21    elif Input.is_action_pressed("ui_down"):
22        velocity.z = SPEED
23    else:
24        velocity.z = lerp(velocity.z, 0.0, 0.1)
25
26    move_and_slide()

```

Tekst 5: Kod skryptu kula.gd.

### 3.3 Śledzenie gracza przez kamerę

Kolejnym etapem było przygotowanie kamery śledzącej ruch gracza. W poradniku wykorzystywano węzeł `InterpolatedCamera`, jednak w wersji silnika Godot 4.6.1 taki element nie był już dostępny. Z tego powodu konieczne było przygotowanie własnego rozwiązania zrealizowanego za pomocą skryptów. W tym celu utworzono dodatkowy węzeł `CameraRig`, którego zadaniem jest płynne podążanie za kulą gracza z zachowaniem ustalonego przesunięcia względem jej położenia. Sam węzeł `Camera3D` nie śledzi więc bezpośrednio obiektu gracza, lecz porusza się razem z `CameraRig`, który pełni rolę pośrednika. Zastosowanie osobnego `CameraRig` pozwoliło uzyskać płynniejsze śledzenie obiektu. W skrypcie `rig` wyznaczana jest pozycja docelowa na podstawie aktualnego położenia kuli oraz zapamiętanego przesunięcia, a następnie węzeł kamery jest stopniowo przesuwany do tej pozycji z użyciem interpolacji liniowej. Dzięki temu kamera nie przeskakuje gwałtownie, lecz porusza się w sposób bardziej naturalny. Dodatkowo w skrypcie przypisanym do `Camera3D` zaimplementowano zmianę lokalnego położenia kamery w zależności od prędkości ruchu gracza. Gdy kula porusza się wolno, kamera pozostaje bliżej obiektu, natomiast przy większej prędkości oddala się.

```

1 extends Node3D
2
3 @onready var target = $"../kula"
4 @export var follow_speed := 5.0
5
6 var offset: Vector3
7
8 func _ready():
9     offset = global_position - target.global_position
10
11 func _physics_process(delta):
12     var desired_position = target.global_position + offset
13     global_position = global_position.lerp(desired_position, min(
        delta * follow_speed, 1.0))

```

Tekst 6: Kod skryptu camera\_rig.gd.

```

1 extends Camera3D
2
3 @onready var target = $"../kula"
4
5 @export var idle_pos := Vector3(0, 3, 5)
6 @export var move_pos := Vector3(0, 4, 8)
7
8 @export var zoom_smoothness := 3.0
9 @export var max_speed_for_zoom := 5.0
10
11 func _physics_process(delta):
12     var horizontal_speed = Vector2(target.velocity.x, target.
        velocity.z).length()
13
14     var speed_ratio = clamp(horizontal_speed / max_speed_for_zoom
        , 0.0, 1.0)
15
16     var desired_pos = idle_pos.lerp(move_pos, speed_ratio)
17
18     position = position.lerp(desired_pos, delta * zoom_smoothness
        )

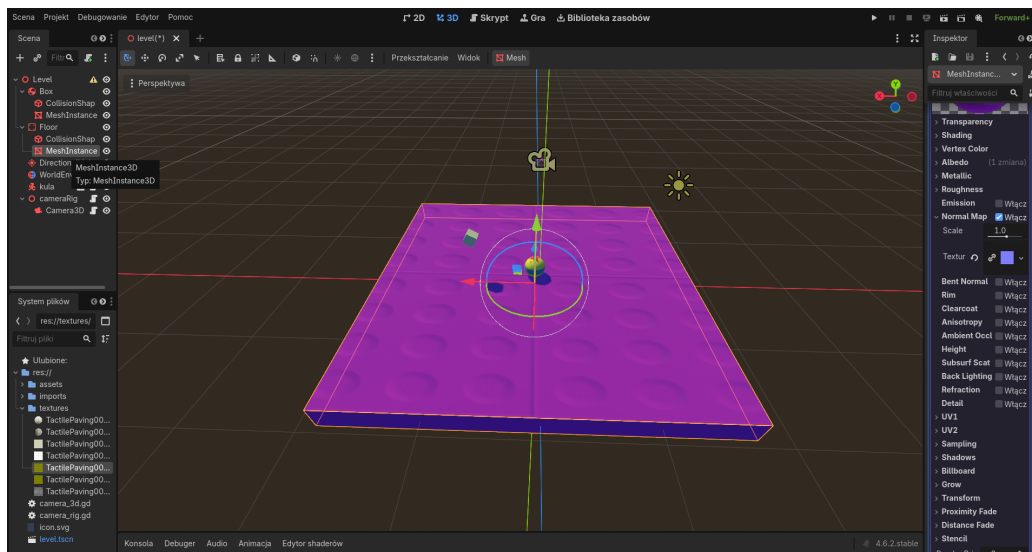
```

Tekst 7: Kod skryptu camera\_3d.gd.

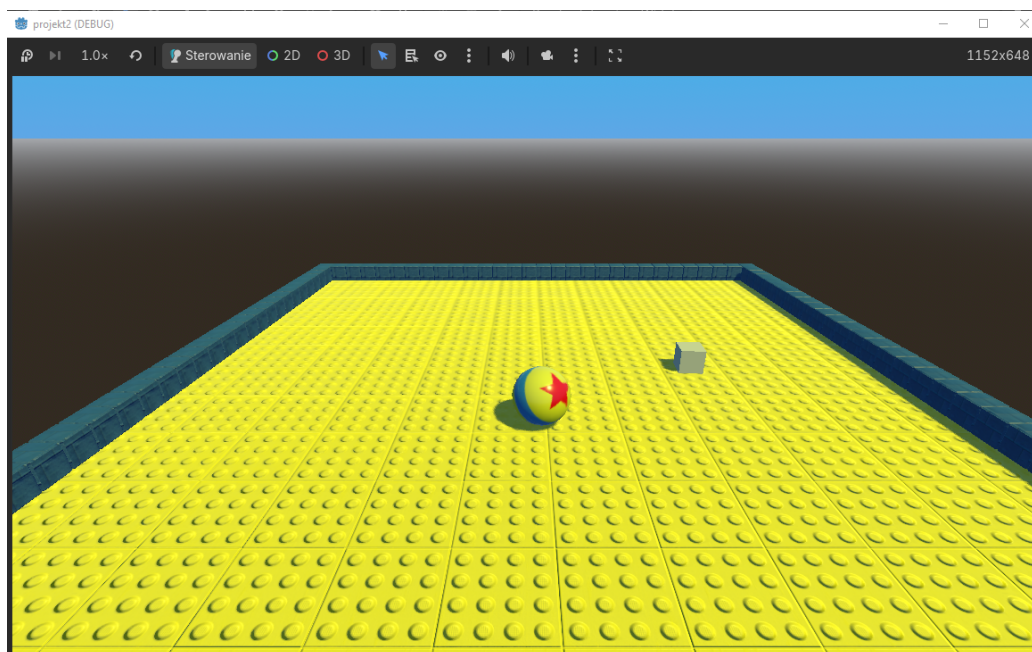
### 3.4 Importowanie tekstur i zmiana wyglądu planszy

Po przygotowaniu podstawowego ruchu gracza oraz mechanizmu śledzenia kuli przez kamerę przystąpiono do poprawy warstwy wizualnej sceny. Na tym etapie zaimportowano tekstury i przypisano je do materiałów wykorzystywanych przez podłogę oraz pozostałe elementy otoczenia. Szczególną uwagę poświęcono wyglądowi planszy. Do materiału podłogi dodano teksturę wzorowaną na powierzchni klocków, a następnie odpowiednio ją przeskalowano, aby wypukłe elementy nie były zbyt duże względem

całej sceny. Zmiana skali tekstury pozwoliła uzyskać drobniejszy, powtarzający się wzór klocków lego. Dodatkowo wykorzystano mapy materiałowe odpowiedzialne za szczegóły powierzchni nadając teksturę lekkiej chropowatości i realizmu.



Rysunek 14: Edytowanie tekstury

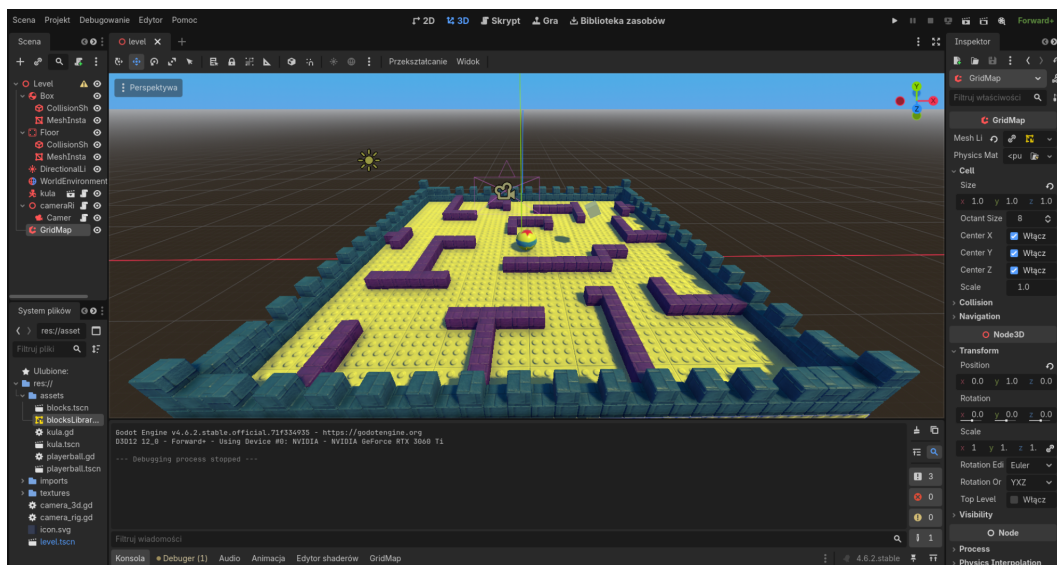


Rysunek 15: Widok planszy po zastosowaniu tekstur



### 3.5 Budowa poziomu z użyciem GridMap

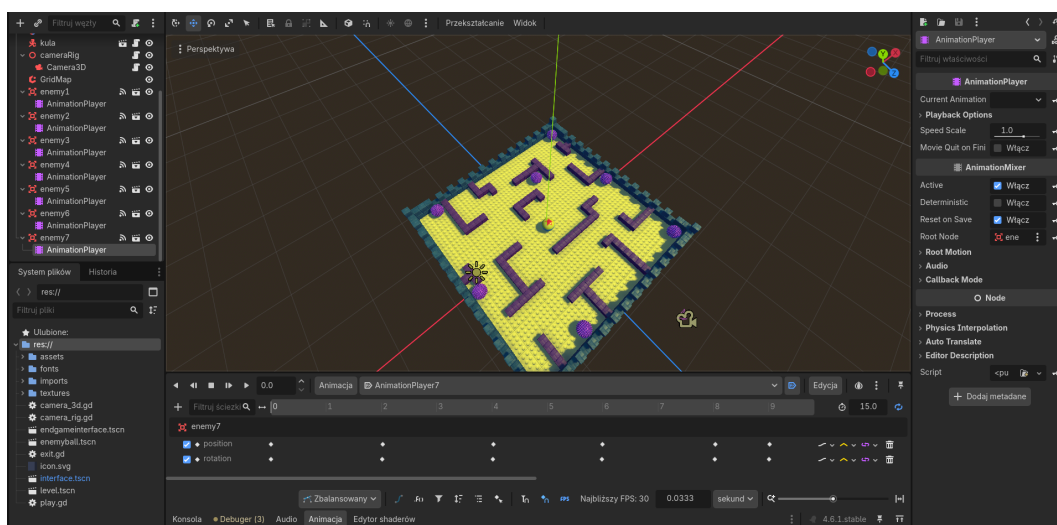
Następnie przygotowano układ poziomu. Do tego celu wykorzystano **GridMap** umożliwiający szybkie rozmieszczanie bloków i budowę ścian planszy służących jako przeszkody dla gracza.



Rysunek 16: Tworzenie planszy przy pomocy GridMap

### 3.6 Przeciwnicy i animacje ruchu

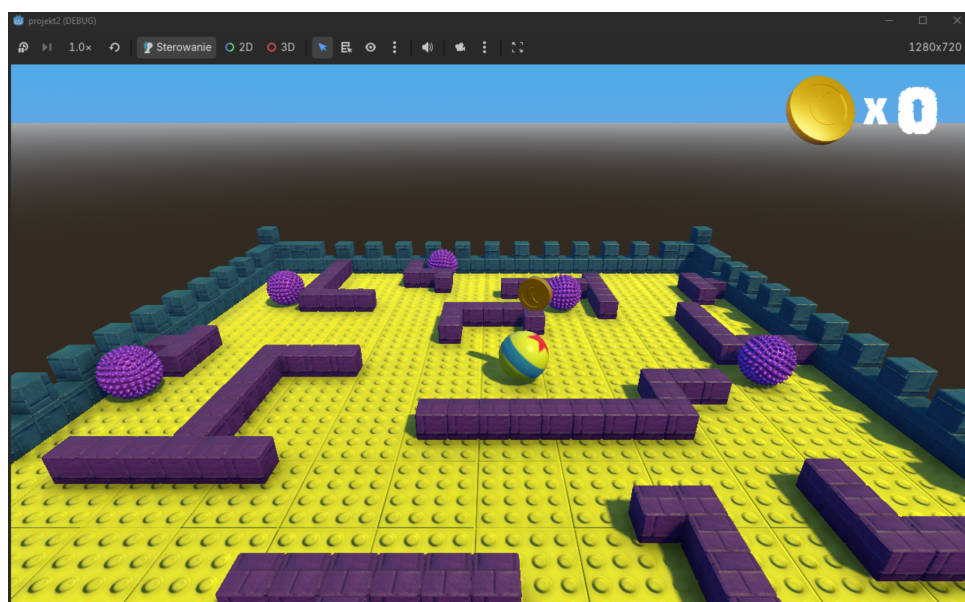
Po przygotowaniu planszy dodano przeciwników jako obiekt **Area3D** poruszających się po określonych torach. Wielokrotnie zduplikowano węzeł zmieniając u każdego z przeciwników schemat poruszania się w **AnimationPlayer**. Ruch wrogich kul został zdefiniowany przy pomocy klatek kluczowych pozycji i rotacji. Dzięki temu przeciwnicy poruszają się po planszy w sposób powtarzalny, stanowiąc przeszkodę dla gracza.



Rysunek 17: Przygotowanie animacji ruchu przeciwników

### 3.7 Dodanie monet, HUD-u oraz ekranów końcowych

W końcowym etapie rozbudowano projekt o elementy odpowiedzialne za przebieg właściwej rozgrywki oraz obsługę interfejsu użytkownika. Na planszy rozmieszczono monety, które gracz może zbierać w trakcie poruszania się po planszy. Każda moneta została wyposażona w prostą animację oraz mechanizm wykrywania kolizji z kulą gracza. Po zebraniu obiektu uruchamiana jest animacja, a następnie moneta znika ze sceny i przekazuje informację o zdobyciu punktu. Przygotowano również HUD wyświetlający aktualną liczbę zebranych monet. W prawym górnym rogu ekranu umieszczono ikonę monety wraz z licznikiem, który jest aktualizowany na bieżąco podczas gry. Po zebraniu wszystkich dziesięciu monet uruchamiany jest licznik czasu, po którym następuje przejście do ekranu zwycięstwa. Przygotowano również osobne ekrany sterujące przebiegiem gry: ekran początkowy wyświetlany po uruchomieniu projektu, ekran porażki pojawiający się po przegranej oraz ekran zwycięstwa wyświetlany po ukończeniu zadania. Każdy z tych interfejsów pozwala rozpocząć nową rozgrywkę lub zakończyć działanie programu.



Rysunek 18: Licznik zebranych monet



Rysunek 19: Przygotowane interfejsy gry

Skrypty przygotowane dla tego etapu odpowiadają za obsługę zbierania monet oraz aktualizację licznika punktów. W skrypcie monety zaimplementowano obrót obiektu, reakcję na wejście gracza w obszar kolizji oraz emisję sygnału po zakończeniu animacji. Z kolei licznik monet aktualizuje wyświetlaną wartość i po osiągnięciu wymaganej liczby zebranych obiektów przełącza grę do ekranu zwycięstwa. Skrypty przypisane do przycisków Play oraz Exit odpowiadają za rozpoczęcie nowej partii i zamknięcie programu.

```

1 extends Area3D
2
3 signal coinCollected
4
5 func _ready() -> void:
6     pass
7
8 func _process(delta: float) -> void:
9     rotate_y(deg_to_rad(1))
10
11 func _on_coin_entered(body: Node3D) -> void:
12     if body.name == "kula":
13         $AnimationPlayer.play("bounce")
14         $Timer.start()
15
16 func _on_timer_timeout() -> void:
17     coinCollected.emit()
18     queue_free()

```

Tekst 8: Kod skryptu coin.gd.

```

1 extends Label
2
3 var coins = 0
4
5 func _ready() -> void:
6     text = str(coins)
7
8 func _process(delta: float) -> void:
9     pass
10
11 func _on_coin_collected() -> void:
12     coins = coins + 1
13     _ready()
14     if coins == 10:
15         $Timer.start()
16
17 func _on_timer_timeout() -> void:
18     get_tree().change_scene_to_file("res://winnerinterface.tscn")

```

Tekst 9: Kod skryptu CoinCounter.gd.

```

1 # play.gd
2 extends Button
3
4 func _ready() -> void:
5     pass
6

```



```

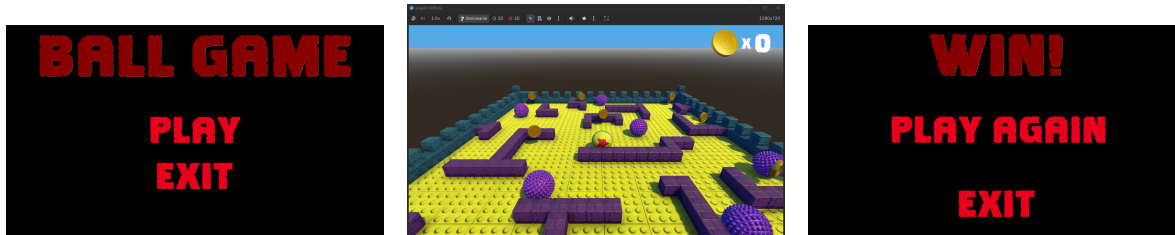
7 func _process(delta: float) -> void:
8     pass
9
10 func _on_play_pressed() -> void:
11     get_tree().change_scene_to_file("res://level.tscn")
12
13
14 # exit.gd
15 extends Button
16
17 func _ready() -> void:
18     pass
19
20 func _process(delta: float) -> void:
21     pass
22
23 func _on_exit_pressed() -> void:
24     get_tree().quit()

```

Tekst 10: Kod skryptów przycisków play.gd oraz exit.gd.

### 3.8 Efekt końcowy projektu 3D

Końcowa wersja gry 3D łączy najważniejsze elementy poznane podczas realizacji tej części laboratorium: przygotowanie sceny 3D, sterowanie graczem, własny system śledzenia kamery, import modeli, użycie materiałów i tekstur, budowę poziomu z wykorzystaniem **GridMap**, animowanych przeciwników, tworzenie HUDu oraz zestawu ekranów interfejsu. Dzięki temu druga część laboratorium stanowiła naturalne rozwinięcie projektu 2D i pokazała, że podobne mechanizmy można zastosować również w przestrzeni trójwymiarowej.



Rysunek 20: Efekt końcowy projektu 3D

## 4 Źródła

1. Oficjalna dokumentacja Godot, *Your First 2D Game*:  
[https://docs.godotengine.org/en/stable/getting\\_started/first\\_2d\\_game/index.html](https://docs.godotengine.org/en/stable/getting_started/first_2d_game/index.html)
2. Poradnik tworzenia gry 3D w Godot, *Creating a Simple 3D Game*  
[https://www.youtube.com/watch?v=VeCrE-ge8xM&list=PLda3VoSoc\\_TSBB0BYwcmlamF1UrjVtccZ](https://www.youtube.com/watch?v=VeCrE-ge8xM&list=PLda3VoSoc_TSBB0BYwcmlamF1UrjVtccZ)